

ASSIGNMENT

10 practical Python questions based on LOOP's. Each question is designed to simulate real-world scenarios, making the concepts relatable and engaging.

Advanced Scenario-Based Loop Questions

1. **Multiplication Table Generator (Nested Loops)** Write a program that generates multiplication tables from 1 to 10. Use a nested loop: one loop for rows (1–10) and another for columns (1–10). Print each row on a new line.

ANSWER:

```
for i in range(1, 11):  
    for j in range(1, 11):  
        print(i * j, end="\t")  
    print()
```

OUTPUT:

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100

2. **Prime Numbers Finder** Write a program to display all prime numbers between 1 and 100. Use a loop to iterate through numbers. Use an inner loop or if statements to check if a number is divisible only by 1 and itself.

ANSWER:

```
for num in range(2, 101):  
    is_prime = True  
  
    for i in range(2, num):  
        if num % i == 0:  
            is_prime = False  
            break
```

```
if is_prime:
    print(num)
```

OUTPUT:

2
3
5
7
11
13
17
19
23
29
31
37
41
43
47
53
59
61
67
71
73
79
83
89
97

3. Seating Arrangement Checker (Nested Loops) Simulate a classroom seating arrangement. There are 3 rows, each with 4 seats. Display a seating chart like: Row 1: Seat 1, Seat 2, Seat 3, Seat 4 Row 2: Seat 1, Seat 2, Seat 3, Seat 4

ANSWER:

```
for row in range(1, 4):
    print(f"Row {row}:", end=" ")
```

```

for seat in range(1, 5):
    print(f"Seat {seat}", end=", ")

print()

```

OUTPUT:

Row 1: Seat 1, Seat 2, Seat 3, Seat 4,

Row 2: Seat 1, Seat 2, Seat 3, Seat 4,

Row 3: Seat 1, Seat 2, Seat 3, Seat 4,

4. Unique Word Counter Write a program to count unique words in a given sentence. Use a loop to split the input string into words. Use an if condition to check if a word already exists in a list.

ANSWER:

```

sentence = input("Enter a sentence: ")

words = sentence.split()
unique_words = []

for word in words:
    word = word.lower()

    if word not in unique_words:
        unique_words.append(word)

print("Unique words count:", len(unique_words))
print("Unique words:", unique_words)

```

OUTPUT:

Enter a sentence: Python is easy and Python is powerful

Unique words count: 5

Unique words: ['python', 'is', 'easy', 'and', 'powerful']

5. Library Book Management A library system has 5 racks with up to 10 books each. Use nested loops to list the books in each rack. Ask the user to search for a book and display which rack it's in.

ANSWER:

```

library = [
    ["Python", "Java", "C", "C++", "SQL", "HTML", "CSS", "JS", "Django",
    "Flask"],

```

```

    ["Maths", "Stats", "Algebra", "Calculus", "Probability", "Trigonometry",
    "Geometry", "R", "SPSS", "Excel"],
    ["AI", "ML", "DL", "Data Science", "Big Data", "NLP", "Power BI",
    "Tableau", "Hadoop", "Spark"],
    ["Economics", "Finance", "Accounts", "Business", "Marketing", "HR",
    "Operations", "Management", "Law", "Tax"],
    ["Physics", "Chemistry", "Biology", "Zoology", "Botany", "Genetics",
    "Microbiology", "Biotech", "Anatomy", "Ecology"]
]

# Display all racks and books
for rack in range(len(library)):
    print(f"Rack {rack + 1}:")
    for book in library[rack]:
        print(" ", book)
    print()

# Search for a book
search_book = input("Enter book name to search: ").strip().lower()
found = False

for rack in range(len(library)):
    for book in library[rack]:
        if book.lower() == search_book:
            print(f"Book found in Rack {rack + 1}")
            found = True
            break
    if found:
        break

if not found:
    print("Book not found in the library")

```

OUTPUT:

Python

Java

C

C++

SQL

HTML

CSS

JS

Django

Flask

Rack 2:

Maths

Stats

Algebra

Calculus

Probability

Trigonometry

Geometry

R

SPSS

Excel

Rack 3:

AI

ML

DL

Data Science

Big Data

NLP

Power BI

Tableau

Hadoop

Spark

Rack 4:

Economics

Finance

Accounts

Business

Marketing
HR
Operations
Management
Law
Tax

Rack 5:

Physics
Chemistry
Biology
Zoology
Botany
Genetics
Microbiology
Biotech
Anatomy
Ecology

Enter book name to search: Python

Book found in Rack 1

6. Pattern Printing (Triangle) Write a program to print the following triangle pattern using nested loops: 1 1 2 1 2 3 1 2 3 4 1 2 3 4 5

ANSWER:

```
for i in range(1, 6):  
    for j in range(1, i + 1):  
        print(j, end=" ")  
    print()
```

OUTPUT:

1
1 2
1 2 3

1 2 3 4

1 2 3 4 5

7. Temperature Comparison Across Cities Simulate a weather app that compares temperatures across 3 cities for 7 days. Use nested loops to iterate over cities and days. Display the highest and lowest temperatures for each city

ANSWER:

```
temperatures = [  
    [30, 32, 31, 29, 28, 33, 34], # City 1  
    [25, 26, 24, 27, 28, 26, 25], # City 2  
    [35, 36, 34, 33, 37, 38, 36]  # City 3  
]  
  
# Iterate through cities  
for city in range(len(temperatures)):  
    highest = temperatures[city][0]  
    lowest = temperatures[city][0]  
  
    # Iterate through days  
    for day in range(len(temperatures[city])):  
        temp = temperatures[city][day]  
  
        if temp > highest:  
            highest = temp  
        if temp < lowest:  
            lowest = temp  
  
    print(f"City {city + 1} → Highest Temp: {highest}, Lowest Temp: {lowest}")
```

OUTPUT:

City 1 → Highest Temp: 34, Lowest Temp: 28

City 2 → Highest Temp: 28, Lowest Temp: 24

City 3 → Highest Temp: 38, Lowest Temp: 33

8. Password Strength Checker Write a program to check the strength of multiple passwords. Use a loop to validate a list of passwords. Use conditions to check: length, special characters, and numbers.

ANSWER:

```
passwords = ["abc123", "Password@123", "hello", "Admin#2024"]  
  
for password in passwords:  
    has_number = False
```

```

has_special = False

# Check each character
for ch in password:
    if ch.isdigit():
        has_number = True
    elif not ch.isalnum(): # Special character
        has_special = True

# Check strength
if len(password) >= 8 and has_number and has_special:
    strength = "Strong"
elif len(password) >= 6 and has_number:
    strength = "Medium"
else:
    strength = "Weak"

print(f"Password: {password} → Strength: {strength}")

```

OUTPUT:

Password: abc123 → Strength: Medium
 Password: Password@123 → Strength: Strong
 Password: hello → Strength: Weak
 Password: Admin#2024 → Strength: Strong

-
9. Inventory Tracker (Nested Loops) Create a program to manage stock levels in a warehouse with multiple sections. Each section has 5 items. Display items running low (<5 units) and their section numbers.

ANSWER:

```

inventory = [
    [10, 3, 7, 2, 8], # Section 1
    [6, 4, 9, 1, 5], # Section 2
    [12, 8, 6, 3, 2] # Section 3
]

# Check inventory
for section in range(len(inventory)):
    print(f"Section {section + 1}:")

    for item in range(len(inventory[section])):
        stock = inventory[section][item]

        if stock < 5:
            print(f"  Item {item + 1} is running low (Stock: {stock})")

    print()

```


OUTPUT:

Section 1:

Item 2 is running low (Stock: 3)

Item 4 is running low (Stock: 2)

Section 2:

Item 2 is running low (Stock: 4)

Item 4 is running low (Stock: 1)

Section 3:

Item 4 is running low (Stock: 3)

Item 5 is running low (Stock: 2)

10. Tic-Tac-Toe Grid Simulate a 3x3 Tic-Tac-Toe grid. Use nested loops to create and display the grid. Allow the user to mark cells (e.g., "X" or "O") by inputting row and column numbers.

ANSWER:

```
grid = [" " for _ in range(3)] for _ in range(3)]

# Function to display the grid
def display_grid():
    for row in range(3):
        for col in range(3):
            print(grid[row][col], end=" | ")
        print("\n-----")

# Display initial grid
display_grid()

# Allow user to mark cells
for turn in range(5): # Max 5 turns for X (simple simulation)
    row = int(input("Enter row (1-3): ")) - 1
    col = int(input("Enter column (1-3): ")) - 1
    mark = input("Enter X or O: ").upper()

    if grid[row][col] == " ":
        grid[row][col] = mark
```

```

else:
    print("Cell already occupied!")

display_grid()

```

OUTPUT: | | |

| | |

| | |

Enter row (1-3): 2

Enter column (1-3): 2

Enter X or O: O

| | |

| O | |

| | |

Enter row (1-3): 1

Enter column (1-3): 2

11. Palindrome Checker Write a program to check if multiple strings from a list are palindromes. Use a loop to iterate through the list. Use if to compare each string with its reverse.

ANSWER:

```

words = ["madam", "hello", "level", "python", "radar"]

for word in words:
    if word == word[::-1]:
        print(word, "is a Palindrome")
    else:

```

```
print(word, "is NOT a Palindrome")
```

OUTPUT:

madam is a Palindrome

hello is NOT a Palindrome

level is a Palindrome

python is NOT a Palindrome

radar is a Palindrome

12. Employee Attendance Tracker Simulate tracking attendance for 10 employees across 5 days. Use nested loops to record attendance (P for present, A for absent). Display a summary of each employee's attendance at the end.

ANSWER:

```
employees = 10
days = 5

# Attendance storage
attendance = []

# Record attendance
for emp in range(employees):
    emp_attendance = []
    print(f"\nEnter attendance for Employee {emp + 1} (P/A):")

    for day in range(days):
        status = input(f"  Day {day + 1}: ").upper()
        emp_attendance.append(status)

    attendance.append(emp_attendance)

# Display attendance summary
print("\n--- Attendance Summary ---")
for emp in range(employees):
    present = 0
    absent = 0

    for day in range(days):
        if attendance[emp][day] == "P":
            present += 1
        else:
            absent += 1

    print(f"Employee {emp + 1}: Present = {present}, Absent = {absent}")
```

OUTPUT:

--- Attendance Summary ---

Employee 1: Present = 4, Absent = 1

Employee 2: Present = 3, Absent = 2

Employee 3: Present = 4, Absent = 1

Employee 4: Present = 3, Absent = 2

Employee 5: Present = 4, Absent = 1

Employee 6: Present = 4, Absent = 1

Employee 7: Present = 4, Absent = 1

Employee 8: Present = 3, Absent = 2

Employee 9: Present = 5, Absent = 0

Employee 10: Present = 2, Absent = 3

13. Grade Categorizer Write a program to categorize student grades. Input grades for 5 students using a loop. Use if conditions to assign letter grades (A, B, C, etc.) and display the results.

ANSWER:

```
for student in range(1, 6):
    marks = int(input(f"Enter marks for Student {student}: "))

    if marks >= 90:
        grade = "A"
    elif marks >= 80:
        grade = "B"
    elif marks >= 70:
        grade = "C"
    elif marks >= 60:
        grade = "D"
    else:
        grade = "F"

    print(f"Student {student} → Grade: {grade}")
```

OUTPUT:

Enter marks for Student 1: 90

Student 1 → Grade: A

Enter marks for Student 2: 80

Student 2 → Grade: B

Enter marks for Student 3: 70

Student 3 → Grade: C

Enter marks for Student 4: 60

Student 4 → Grade: D

Enter marks for Student 5: 50

Student 5 → Grade: F

14. Pyramid Pattern (Numbers) Write a program to print the following pattern: 1 1
2 1 2 3 1 2 3 4

ANSWER:

```
for i in range(1, 5):  
    for j in range(1, i + 1):  
        print(j, end=" ")  
    print()
```

OUTPUT: 1

1 2

1 2 3

1 2 3 4

15. Sales Data Analysis Analyze sales data for 3 products over 4 months. Use nested loops to store and display sales data. Highlight the product with the highest sales each month.

ANSWER:

```
sales = [  
    [1200, 1500, 1800, 2000], # Product 1  
    [1000, 1700, 1600, 1900], # Product 2  
    [1300, 1400, 1750, 2100]  # Product 3  
]  
  
products = 3  
months = 4
```

```

# Display sales data
print("Sales Data:")
for p in range(products):
    print(f"Product {p + 1}:", end=" ")
    for m in range(months):
        print(sales[p][m], end=" ")
    print()

print("\nHighest Sales Each Month:")

# Find highest sales each month
for m in range(months):
    highest = sales[0][m]
    top_product = 1

    for p in range(1, products):
        if sales[p][m] > highest:
            highest = sales[p][m]
            top_product = p + 1

    print(f"Month {m + 1}: Product {top_product} → Sales = {highest}")

```

OUTPUT:

Highest Sales Each Month:

Month 1: Product 3 → Sales = 1300

Month 2: Product 2 → Sales = 1700

Month 3: Product 1 → Sales = 1800

Month 4: Product 3 → Sales = 2100

16. Palindrome Numbers in a Range Write a program to find all palindrome numbers between 10 and 200. Use a loop to iterate through the range. Use if conditions to check if a number reads the same backward as forward.

ANSWER:

```

for num in range(10, 201):
    temp = num
    reverse = 0

    # Reverse the number
    while temp > 0:
        digit = temp % 10
        reverse = reverse * 10 + digit
        temp //= 10

    # Check palindrome

```

```
if num == reverse:
    print(num)
```

OUTPUT:

11
22
33
44
55
66
77
88
99
101
111
121
131
141
151
161
171
181
191

17. Magic Square Validator Write a program to check if a given 3x3 matrix is a magic square (all rows, columns, and diagonals add up to the same number). Use nested loops to iterate through the matrix. Use if conditions to validate the sums.

ANSWER:

```
matrix = []
print("Enter elements of 3x3 matrix:")

for i in range(3):
    row = []
    for j in range(3):
        row.append(int(input(f"Element [{i+1}][{j+1}]: ")))
```

```

        matrix.append(row)

# Calculate magic sum (sum of first row)
magic_sum = 0
for j in range(3):
    magic_sum += matrix[0][j]

is_magic = True

# Check row sums
for i in range(3):
    row_sum = 0
    for j in range(3):
        row_sum += matrix[i][j]
    if row_sum != magic_sum:
        is_magic = False

# Check column sums
for j in range(3):
    col_sum = 0
    for i in range(3):
        col_sum += matrix[i][j]
    if col_sum != magic_sum:
        is_magic = False

# Check diagonals
diag1 = 0
diag2 = 0
for i in range(3):
    diag1 += matrix[i][i]
    diag2 += matrix[i][2 - i]

if diag1 != magic_sum or diag2 != magic_sum:
    is_magic = False

# Result
if is_magic:
    print("The matrix is a Magic Square")
else:
    print("The matrix is NOT a Magic Square")

```

OUTPUT:

Enter elements of 3x3 matrix:

Element [1][1]: 8

Element [1][2]: 1

Element [1][3]: 6

Element [2][1]: 3

Element [2][2]: 5

Element [2][3]: 7

Element [3][1]: 4

Element [3][2]: 9

Element [3][3]: 2

The matrix is a Magic Square

18. Meal Planner Simulate a meal planner for a week. Use a nested loop to list 7 days and 3 meals (breakfast, lunch, dinner). Allow the user to input or modify meals for each day

ANSWER:

```
days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday",
"Sunday"]
meals = ["Breakfast", "Lunch", "Dinner"]

meal_plan = []

# Input meals
for day in range(7):
    print(f"\nEnter meals for {days[day]}:")
    day_meals = []

    for meal in range(3):
        food = input(f"    {meals[meal]}: ")
        day_meals.append(food)

    meal_plan.append(day_meals)

# Display meal plan
print("\n--- Weekly Meal Plan ---")
for day in range(7):
    print(days[day] + ":")
    for meal in range(3):
        print(f"    {meals[meal]} → {meal_plan[day][meal]}")
```

OUTPUT:

Enter meals for Monday:

Breakfast: SALAD

Lunch: RICE

Dinner: ROTI

Enter meals for Tuesday:

Breakfast: SALAD

Lunch: RICE

Dinner: ROTI

Enter meals for Wednesday:

Breakfast: SAME

Lunch: SAME

Dinner: SAME

Enter meals for Thursday:

Breakfast: SAME

Lunch: SAME

Dinner: SAME

Enter meals for Friday:

Breakfast: SA,E

Lunch: SAME

Dinner: SAME

Enter meals for Saturday:

Breakfast: SANDWITCH

Lunch: DOSA

Dinner: NAAN

Enter meals for Sunday:

Breakfast: OATS

Lunch: CHOLE BHATURE

Dinner: LASSI

--- Weekly Meal Plan ---

Monday:

Breakfast → SALAD

Lunch → RICE

Dinner → ROTI

Tuesday:

Breakfast → SALAD

Lunch → RICE

Dinner → ROTI

Wednesday:

Breakfast → SAME

Lunch → SAME

Dinner → SAME

Thursday:

Breakfast → SAME

Lunch → SAME

Dinner → SAME

Friday:

Breakfast → SA,E

Lunch → SAME

Dinner → SAME

Saturday:

Breakfast → SANDWITCH

Lunch → DOSA

Dinner → NAAN

Sunday:

Breakfast → OATS

Lunch → CHOLE BHATURE

Dinner → LASSI